SE-AFTER-MID — Weeks 9–15 · 12 Oct – 29 Nov 2026 · FINAL (W15)

Scope: Weeks 9–15 (12 Oct – 29 Nov 2026) — verification & validation, white-box testing, black-box testing & SQA, COCOMO cost estimation, software maintenance, final taper, ending at the Week 15 final + lab viva. **Siblings:** `SE-NAV.md` (master map) · `SE-BEFORE-MID.md` (Weeks 1–8) · `SE-LABS.md` (labs). Week-colored headers below match the NAV schedule. **Exam split:** ~60% theory + ~40% numerical/diagram, final-weighted toward COCOMO, testing, and design patterns (80%/75%/70% final prob — see NAV). COCOMO numericals are the free marks. **Exam order:** COCOMO numericals → DFD/cyclomatic → theory (final). Lab viva in a separate W15 slot.

At a glance (W9–W15)

W	Dates	Variant	File	Topics
W9	12–18 Oct	P1	AFTER	Verification & validation (recovery)
W10	19–25 Oct	P2	AFTER	White-box testing — cyclomatic
W11	26 Oct–01 Nov	P2	AFTER	Black-box testing & SQA
W12	02–08 Nov	P2	AFTER	COCOMO — cost estimation
W13	09–15 Nov	P1	AFTER	Software maintenance
W14	16–22 Nov	P0	AFTER	Final taper — past paper FIRST
W15	23–29 Nov	FINAL	AFTER	Exam + lab viva — execute

Tier key: **P0** = no exam pressure (front-load new material) · **P1** = light revision · **P2** = drill-heavy week. Variants editorial (SE manifests carry no variant field).

W9 — Verification & Validation · 12–18 Oct · P1 · 5 hrs

Banner: Rotation SE (Thu+Fri) · Tier P1 · Time budget 5 hrs · P0 floor: pack pass + def/formula skim + same-problem drill. **Recovery week — ledger MUST clear to 0.**

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 9	lines 165–184
Fear-Killer Pack	`Fear-Killer-Packs.md` → Week 6: topic-verification-and-validation	lines 47–52
Week manifest	`weeks/SE-W9.md`	full file
Chapter breakdown	— (no `03-Chapter-Breakdowns/`; narrative-derived topic)	—

Definitions (verbatim)

Term	Definition
Verification	"Are we building the product right?" — checking that the product meets specifications.
Validation	"Are we building the right product?" — checking that the product meets customer needs.

Formulas (verbatim)

Formula	Statement
—	(none in `Formula-Book.md` this week)

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#20** V-Model for testing
Numericals	— (none; V&V blank-page + DFD interleaved drill)
Tricky	— (deferred)
Top-100	— (deferred)

Books · Chapters · Media

Resource	Where
Pressman & Maxim	Ch.14
Gate Smashers	Testing video

Fear-Killer Pack SE-W9 (verbatim — Q3)

Resources: Gate Smashers (Testing videos) • Pressman Ch.14-16 • Sommerville Ch.8

3. Differentiate verification and validation. Given a requirements document for a railway signaling system, describe what you would do for verification vs validation. Then differentiate unit, integration, system, and acceptance testing for the same system. Who performs each type and when?

> Full pack `Week 6: topic-verification-and-validation` at `Fear-Killer-Packs.md` L47–52. Q1 (cyclomatic) returns at W10; Q2 (EP/BVA) returns at W11.

Narrative — Week 9 (verbatim)

Topics: V&V fundamentals; Software inspection; Testing fundamentals; Verification vs validation; Debugging

Resources: Pressman Ch.14; Gate Smashers — Testing

Recovery note: If midterm scored below 85%, take 2 days at reduced load (20 hrs/week total). Recalibrate — a 78% midterm with 95%+ finals still yields B+/A-.

Practice: - Verification vs validation: differentiate from blank page (V = "are we building it right?", V = "are we building the right thing?") - Inspection process: roles and steps — retrieve from memory - Debugging strategies (brute force, backtracking, cause elimination) — closed book comparison - **Interleaved retrieval (20 min):** Draw context + Level 0 + Level 1 DFD for a new scenario from blank paper. Verify balance across levels. All closed book.

Retrieval protocol: Every practice item starts from a blank page. No peeking at notes.

Lab: Begin test case design for your project

Time budget: 5 hrs

Lab pointer

Lab this week: `SE-LABS.md` → **Lab 8 — Program testing in SDLC** (test case design; derived map — no `Lab-Schedule.md`).

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q3 V-vs-V + test levels) · def skim (Verification/Validation) · same-problem drill (V&V blank-page + 2 DFD examples)
- **Same-problem drill target:** V&V distinction < 5 min; inspection roles/steps from memory; 2 complete DFD examples closed book
- **Deliverable:** begin test case design for project
- **Trap:** recovery mandate — ledger MUST clear to 0 before Sunday night; V vs V mnemonic = "building it right?" vs "building the right thing?"

W10 — White-Box Testing · 19–25 Oct · P2 · 5 hrs

Banner: Rotation SE (Thu+Fri) · Tier P2 · Time budget 5 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref

Narrative	`Week-by-Week-Narrative.md` → Week 10	lines 187–204
Fear-Killer Pack	`Fear-Killer-Packs.md` → Week 6: topic-verification-and-validation (Q1)	lines 47–52 (Q1 L50)
Week manifest	`weeks/SE-W10.md`	full file
Chapter breakdown	— (no `03-Chapter-Breakdowns/`; narrative-derived topic)	—

Definitions (verbatim)

Term	Definition
Cyclomatic Complexity	A software metric that measures the number of linearly independent paths through a program's source code.

Formulas (verbatim)

Formula	Statement
Cyclomatic Complexity	M = E – N + 2P (Edges – Nodes + 2 × Connected Components)
Cyclomatic Complexity	M = Number of Predicate Nodes + 1

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#14** Control flow graph (for cyclomatic complexity)
Numericals	`Numerical-Book.md` **#20** Cyclomatic complexity = E – N + 2P · **#21** Statement/branch/path coverage percentage
Tricky	— (deferred)
Top-100	— (deferred)

Books · Chapters · Media

Resource	Where
Pressman & Maxim	Ch.14
Past papers	white-box problems

Fear-Killer Pack SE-W10 (verbatim — Q1)

Resources: Gate Smashers (Testing videos) • Pressman Ch.14-16 • Sommerville Ch.8

1. Given the following code: `int f(int a, int b, int c) { if (a > 5) { if (b < 10) { return c / (b - a); } } return 0; }` Draw the control flow graph. Calculate cyclomatic complexity using all three methods (edges-nodes+2, regions, predicates+1). Design test cases to achieve 100% statement coverage, 100% branch coverage, and 100% path coverage. How many test cases for each?

Narrative — Week 10 (verbatim)

Topics: White-box testing techniques; Statement coverage; Branch coverage; Path coverage; Cyclomatic complexity (M = E – N + 2P)

Resources: Pressman Ch.14; past paper problems

Practice: - Draw control flow graph from code — closed book - Compute cyclomatic complexity for 5 functions — retrieve formula from memory - Generate test cases for statement, branch, path coverage — from blank page - **Interleaved retrieval (20 min):** Process model comparison table from memory — Waterfall, Incremental, Spiral, Agile — list phases, when to use, when NOT to use. All closed book.

Killer trap: Cyclomatic complexity formula — every term matters. E = edges, N = nodes, P = connected components. Most students forget P.

Deliverable: 5 cyclomatic complexity calculations with test cases

Time budget: 5 hrs

Lab pointer

No lab milestone this week (narrative W10 has no `**Lab:**` line; natural Lab 8 alignment is heuristic and unconfirmed —

omitted per MASTER-DECISIONS Tier 13 #5).

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 CFG + cyclomatic + coverage) · def/formula skim (Cyclomatic Complexity + M = E–N+2P) · same-problem drill #20/#21
- **Same-problem drill target:** CFG draw < 15 min; cyclomatic via all three methods (E–N+2P, regions, predicates+1); coverage test counts
- **Deliverable:** 5 cyclomatic complexity calculations with test cases
- **Trap:** every term matters — E = edges, N = nodes, P = connected components; **most students forget P** (killer trap)

W11 — Black-Box Testing & SQA · 26 Oct–01 Nov · P2 · 5 hrs

Banner: Rotation SE (Thu+Fri) · Tier P2 · Time budget 5 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 11	lines 207–224
Fear-Killer Pack	`Fear-Killer-Packs.md` → Week 6 (Q2) + Week 8: topic-maintenance-and-sqa (Q3)	L47–52 (Q2 L51) · L62–67 (Q3 L67)
Week manifest	`weeks/SE-W11.md`	full file
Chapter breakdown	— (no `03-Chapter-Breakdowns/`; narrative-derived topic)	—

Definitions (verbatim)

Term	Definition
Equivalence Partitioning	A black-box testing technique that divides input data into partitions of equivalent data.

Formulas (verbatim)

Formula	Statement
—	(none in `Formula-Book.md` this week; EP/BVA is procedural)

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#20** V-Model (testing levels)
Numericals	— (none; drill = equivalence classes + BVA boundary set for a date-range function)
Tricky	— (deferred)
Top-100	— (deferred)

Books · Chapters · Media

Resource	Where
Pressman & Maxim	Ch.14 (black-box), Ch.15 (SQA)

Fear-Killer Pack SE-W11 (verbatim — Q2 + Q3)

Resources (Week 6 Q2): Gate Smashers (Testing videos) • Pressman Ch.14-16 • Sommerville Ch.8

2. Apply equivalence partitioning and boundary value analysis to a function that accepts a date in format DD/MM/YYYY. The valid range is 01/01/2000 to 31/12/2099. Identify all equivalence classes (valid and invalid) and boundary values. Generate the minimal test set for BVA including all boundaries.

Resources (Week 8 Q3): Pressman Ch.13, 17 • Sommerville Ch.9, 24

3. Define SQA and list 5 SQA activities. For a critical medical device software, design the SQA plan including: standards to

follow, review types (technical, management), testing levels, and metrics to track. How does SQA differ from testing? Given a budget of 15% of total project cost, allocate the SQA budget across activities.

Narrative — Week 11 (verbatim)

Topics: Black-box testing: equivalence partitioning, boundary value analysis; SQA basics; Software metrics; Statistical SQA

Resources: Pressman Ch.14 (black-box), Ch.15 (SQA)

Practice: - Equivalence partitioning: divide input domain into valid/invalid classes — closed book - Boundary value analysis: test at boundaries, not middle values — retrieve from memory - Software metrics: McCabe vs Halstead — blank page comparison - **Interleaved retrieval (20 min):** Scrum roles and ceremonies from memory — Product Owner vs Scrum Master vs Dev Team, sprint artifacts, ceremony purposes. Then design pattern comparison (Singleton, Factory, Observer). All closed book.

Lab: Execute test cases on your project, document results

Deliverable: Black-box test cases for 3 modules of your project

Time budget: 5 hrs

Lab pointer

Lab this week: `SE-LABS.md` → **Lab 8 — Program testing in SDLC** (execute test cases + document; derived map — no `Lab-Schedule.md`).

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q2 EP/BVA for DD/MM/YYYY + Q3 SQA plan) · def skim (Equivalence Partitioning) · same-problem drill (EP + BVA boundary set)
- **Same-problem drill target:** EP valid/invalid classes < 10 min; BVA boundary set (min, min–1, max, max+1, nominal) closed book
- **Deliverable:** black-box test cases for 3 modules of your project
- **Trap:** BVA tests boundaries, not middle values; SQA vs testing is a different concept — SQA is process-level, testing is product-level

W12 — Software Cost Estimation (COCOMO) · 02–08 Nov · P2 · 5 hrs

Banner: Rotation SE (Thu+Fri) · Tier P2 · Time budget 5 hrs · P0 floor: pack pass + def/formula skim + same-problem drill. **Start sleep banking: 9 hrs (bedtime 22:00) for 7 nights.**

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 12	lines 227–251
Fear-Killer Pack	`Fear-Killer-Packs.md` → Week 7: topic-cost-estimation	lines 54–60
Week manifest	`weeks/SE-W12.md`	full file
Chapter breakdown	— (no `03-Chapter-Breakdowns/`; narrative-derived topic)	—

Definitions (verbatim)

Term	Definition
COCOMO	Constructive Cost Model — an empirical model for estimating software project effort, schedule, and cost.

Formulas (verbatim)

Formula	Statement
Effort	Effort (PM) = a × (KLOC)^b
Schedule	Schedule (TDEV) = c × (Effort)^d
Team Size	Team Size = Effort / Schedule
Intermediate COCOMO	Effort = a × (KLOC)^b × EAF (Effort Adjustment Factor)

COCOMO coefficients (flashcard these — killer trap)

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#19** COCOMO model breakdown
Numericals	`Numerical-Book.md` **#16** Basic COCOMO effort · **#17** Schedule · **#18** Team size · **#19** Intermediate COCOMO with EAF
Tricky	— (deferred)
Top-100	— (deferred)

Books · Chapters · Media

Resource	Where
Pressman & Maxim	Ch.23
COCOMO	example problems

Fear-Killer Pack SE-W12 (verbatim)

Resources: Gate Smashers (COCOMO videos) • Pressman Ch.23

1. Calculate effort, schedule, and team size using Basic COCOMO for an Organic project of 50 KLOC. Use coefficients: a=2.4, b=1.05, c=2.5, d=0.38. Then recalculate for a Semi-detached project (a=3.0, b=1.12, c=2.5, d=0.35). Compare the two — which requires more effort and why?
2. Using Intermediate COCOMO, the nominal effort for 40 KLOC Semi-detached is 160 PM. The project has: high required reliability (1.15), high product complexity (1.15), low analyst capability (1.29), low language experience (1.14), and high use of modern tools (0.91). Calculate EAF and adjusted effort. Which cost driver has the most impact?
3. Differentiate size-oriented metrics (LOC) from function-oriented metrics (FP). Given a project with 10,000 LOC in Java delivering 200 function points at 12 person-months, calculate: productivity (LOC/PM and FP/PM), quality (defects/KLOC), and cost per FP (at $8000/PM). Why might FP be preferred over LOC?
4. Calculate the exponents and coefficients for Basic COCOMO: if effort = 2.4 × (KLOC)^1.05 for Organic, what is the effort and schedule for 100 KLOC? Show the formulas and then explain why the exponent > 1 implies diseconomy of scale.

Narrative — Week 12 (verbatim)

Topics: Basic, Intermediate, Detailed COCOMO; Mode selection (Organic, Semi-detached, Embedded); Effort adjustment factors

Resources: Pressman Ch.23; COCOMO example problems

Sleep banking: Begin sleeping 9 hrs (bedtime 22:00) for 7 nights leading to finals. This inoculates against exam-night sleep loss.

Practice: - **Memorize COCOMO coefficients (retrieval practice):** Write all 12 coefficients from memory — Organic, Semi-detached, Embedded — before checking the table. Repeat until 100% accurate. - Organic: a=2.4, b=1.05, c=2.5, d=0.38 - Semi-detached: a=3.0, b=1.12, c=2.5, d=0.35 - Embedded: a=3.6, b=1.20, c=2.5, d=0.32 - Solve 5 complete COCOMO problems: effort, schedule, team size — closed book - Intermediate COCOMO: apply 15 cost drivers (EAF) — retrieve adjustment factors from memory - **Interleaved retrieval (20 min):** Draw context + Level 0 DFD from blank paper for a new scenario. Then compute cyclomatic complexity for a sample flow graph. All closed book.

Killer trap: One wrong coefficient lookup creates chain errors through the entire estimate. Flashcard every coefficient until reflex-level.

Lab: Apply COCOMO to your project, include in final report

Deliverable: COCOMO calculation for your project

Time budget: 5 hrs

Lab pointer

Lab this week: `SE-LABS.md` — **no direct workbook lab maps** (COCOMO is theory). Deliverable only: apply COCOMO to your project for the final report (narrative W12 **Lab**, L246).

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 Basic Organic-vs-Semi-detached + Q2 EAF) · def/formula skim (COCOMO + all formulas) · same-problem drill #16–#18
- **Same-problem drill target:** all 12 coefficients from memory, 100% accurate; complete COCOMO problem (effort + schedule + team size) < 15 min closed book
- **Deliverable:** COCOMO calculation for your project
- **Trap:** one wrong coefficient → chain errors through the entire estimate — flashcard every coefficient (12 total) until reflex-level (killer trap); sleep banking 9 h starts now

---

W13 — Software Maintenance · 09–15 Nov · P1 · 4 hrs

Banner: Rotation SE (Thu+Fri) · Tier P1 · Time budget 4 hrs · P0 floor: pack pass + def/formula skim + same-problem drill.

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 13	lines 254–269
Fear-Killer Pack	`Fear-Killer-Packs.md` → Week 8: topic-maintenance-and-sqa	lines 62–67
Week manifest	`weeks/SE-W13.md`	full file
Chapter breakdown	— (no `03-Chapter-Breakdowns/`; narrative-derived topic)	—

Definitions (verbatim)

Term	Definition
—	(no new definitions this week — reuse `Definition-Book.md`; Verification/Validation for V-Model context)

Formulas (verbatim)

Formula	Statement
—	(none in `Formula-Book.md` this week)

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	— (none specific; maintenance process model diagram drawn from memory)
Numericals	— (none; drill = maintenance-type identification from scenarios)
Tricky	— (deferred)
Top-100	— (deferred)

Books · Chapters · Media

Resource	Where
Pressman & Maxim	Ch.13/17
Past papers	maintenance theory questions

Fear-Killer Pack SE-W13 (verbatim — Q1 + Q2)

Resources: Pressman Ch.13, 17 • Sommerville Ch.9, 24

1. A banking application has 5 million LOC with 500 reported defects per year. The maintenance team handles 2000 change requests annually: 40% corrective, 25% adaptive, 20% perfective, 15% preventive. Calculate the maintenance effort distribution and compute the maintenance cost if each PM costs $10,000 and the team processes 50 PM per month. Show the cost breakdown by maintenance type.
2. Compare the four types of software maintenance. Take a legacy payroll system: the tax laws change (adaptive), a bug causes incorrect overtime calculation (corrective), the UI needs modernization (perfective), and code documentation needs updating (preventive). Show the effort distribution and which type typically consumes the most resources.

Full pack `Week 8: topic-maintenance-and-sqa` at `Fear-Killer-Packs.md` L62–67. Q3 (SQA) was used at W11.

Narrative — Week 13 (verbatim)

Topics: Types of maintenance (corrective, adaptive, perfective, preventive); Maintenance process models; Reverse engineering; Re-engineering

Resources: Pressman Ch.13/17; past paper theory questions

Practice: - Maintenance type identification: given a scenario, identify which type — closed book - Maintenance process model diagram — retrieve from memory - Re-engineering vs reverse engineering: distinction — blank page comparison - **Interleaved retrieval (20 min):** COCOMO speed drill — all 12 coefficients from memory, then solve 2 complete problems in 15 min. All closed book.

Lab: Finalize project report, complete all documentation

Time budget: 4 hrs

Lab pointer

Lab this week: `SE-LABS.md` — **no direct workbook lab maps** (documentation completion). Deliverable only: finalize project report (SRS, design, testing, COCOMO sections) per narrative W13 **Lab**, L266.

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** fear-killer pack pass (Q1 effort distribution + Q2 four types) · def skim (none new) · same-problem drill (maintenance-type identification)
- **Same-problem drill target:** maintenance-type identification < 5 min per scenario; re-engineering-vs-reverse-engineering comparison from memory
- **Deliverable:** project report finalized (SRS, design, testing, COCOMO sections)
- **Trap:** light theory week — protects sleep banking for finals; COCOMO coefficients must survive via interleaved retrieval

---

W14 — Final Exam Preparation & Project Submission · 16–22 Nov · P0 · 7 hrs

Banner: Rotation SE (Thu+Fri) · Tier P0 · Time budget 7 hrs (incl. project wrap-up) · P0 floor: pack pass (revision-cycle-sourced) + def/formula skim + same-problem drill. **START with the past paper, NOT with review.**

Sources & offsets

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 14	lines 272–293
Fear-Killer Pack	— (taper — no pack; re-run SE-W1/2/4/5/6/7/8)	—
Week manifest	`weeks/SE-W14.md`	full file
Chapter breakdown	— (all W1–W13 topics)	—

Definitions (verbatim) — full W1–W13 sweep

Term	Definition
Agile	A software development methodology based on iterative development, where requirements and solutions evolve through collaboration.
Cohesion	The degree to which elements within a module belong together (high cohesion is good).
COCOMO	Constructive Cost Model — an empirical model for estimating software project effort, schedule, and cost.
Coupling	The degree of interdependence between modules (low coupling is good).
Cyclomatic Complexity	A software metric that measures the number of linearly independent paths through a program's source code.
DFD (Data Flow Diagram)	A graphical representation of data flow through a system, showing processes, data stores, and external entities.
Equivalence Partitioning	A black-box testing technique that divides input data into partitions of equivalent data.

Functional Requirement	A requirement that specifies what a system must do (functions, features).
Non-Functional Requirement	A requirement that specifies constraints on the system (performance, security, usability).
Scrum	An agile framework for managing software development with sprints, roles (Product Owner, Scrum Master, Dev Team), and ceremonies.
SRS (Software Requirements Specification)	A document that describes what the software will do and how it will behave.
Verification	"Are we building the product right?" — checking that the product meets specifications.
Validation	"Are we building the right product?" — checking that the product meets customer needs.
WBS (Work Breakdown Structure)	A hierarchical decomposition of the total scope of work.

Formulas (verbatim) — full sweep

Formula	Statement
Cyclomatic Complexity	M = E – N + 2P · M = predicates + 1
Effort	Effort (PM) = a × (KLOC)^b
Schedule	Schedule (TDEV) = c × (Effort)^d
Team Size	Team Size = Effort / Schedule
Intermediate COCOMO	Effort = a × (KLOC)^b × EAF

Diagrams · Numericals · Tricky · Top-100

Type	Items
Diagrams	`Diagram-Book.md` **#1–#20** full sweep (process models, DFD, WBS, Gantt, UML, CFG, patterns, Scrum, COCOMO, V-Model)
Numericals	`Numerical-Book.md` **#16–#21** (COCOMO + cyclomatic + coverage)
Tricky	— (deferred)
Top-100	— (deferred)

Books · Chapters · Media

Resource	Where
Weeks 1–13 packs	timed past papers

Fear-Killer Pack — final taper (all packs)

No single pack this week. Re-run every pack under timed conditions: SE-W1/2/4/5 (pre-mid) + SE-W6/7/8 (post-mid).

Narrative — Week 14 (verbatim)

Topics: Cumulative review Weeks 1-13; Past paper practice; Project finalization

Practice: - Timed full-length past paper (3 hrs, closed book) — start with the past paper, NOT with review - COCOMO numerical speed drills (5 in 30 min) — retrieve coefficients from memory each time - DFD drawing: 2 complete examples — blank page, closed book - Cyclomatic complexity: 5 problems — retrieve formula from memory - Theory: process models, Scrum, SQA, maintenance — forced recall via blank page outlines - **Spaced repetition:** Redrill any COCOMO coefficients or DFD patterns that took >2 min

Retrieval protocol: No passive reading at all this week. Every minute is active retrieval.

Sleep banking: Continue 9 hrs sleep (bedtime 22:00). Do NOT trade sleep for last-minute studying — it degrades recall by 15-20% per lost hour.

Important: Submit final project deliverable this week (before final exam)

Deliverable: Final project submission + one solved past paper

Time budget: 7 hrs (including project wrap-up)

Lab pointer

Lab this week: `SE-LABS.md` — **final project submission (lab-grade close-out)**. No `**Lab:**` line in narrative W14 — the final submission (narrative Important, L288) is the lab-grade close-out; project + all reports finalized.

P0 floor · Drill target · Deliverable · Trap

- **P0 floor:** revision-cycle checklist (COCOMO coefficients, DFD, cyclomatic, process models, Scrum, SQA, maintenance — blank page) · def/formula skim (all W1–W13; formulas from memory first) · same-problem drill #16–#21 speed drills
- **Same-problem drill target:** COCOMO 5-in-30-min; cyclomatic 5 problems; answer order on paper = COCOMO numericals → DFD/cyclomatic → theory (5-point structure: definition, explanation, example, advantage, disadvantage)
- **Deliverable:** final project submission + one solved past paper
- **Trap:** no passive reading — every minute active retrieval; START with the past paper, NOT with review; do NOT trade sleep (recall degrades 15–20% per lost hour); submit final project deliverable this week (before final exam)

W15 — FINAL EXAM & LAB VIVA · 23–29 Nov · Exam

Banner: Rotation SE (Thu+Fri) · Tier FINAL · No new deep study, no floor accrual. Ledger frozen during W15.

Sources

Source	Where	Ref
Narrative	`Week-by-Week-Narrative.md` → Week 15	lines 296–309
Pack	`Fear-Killer-Packs.md` → Week 15	— (no pack)
Week manifest	(W15 note at `weeks/SE-W14.md` L58–60)	reference

Fear-Killer Pack — Week 15 (verbatim)

Week 15: FINAL EXAM — No new pack. Execution only: COCOMO numericals → DFD/cyclomatic → theory; write every step of every numerical.

Narrative — Week 15 (verbatim)

Focus: Execution. Do not learn anything new.

Theory Exam: - Review 1-page cheat sheet (modes, coefficients, process model table) - Answer order: COCOMO numericals → DFD/cyclomatic → theory - For theory questions: use 5-point structure (definition, explanation, example, advantage, disadvantage)

Lab Viva (separate slot): - Prepare 3-min verbal walkthrough of your project: problem → design → implementation → testing → lessons learned - Anticipate 10 most likely questions (see viva Q bank) - Know your SRS and UML diagrams cold - Dress professionally. Speak clearly. Own your project.

Exam-day stack (final) + viva

- [ ] Theory exam order: COCOMO numericals → DFD/cyclomatic → theory; theory = 5-point structure (definition, explanation, example, advantage, disadvantage)
- [ ] Lab viva (separate slot): 3-min project walkthrough → 10 technical questions; `SE-LABS.md` viva + `Viva-Book.md`
- [ ] Sleep 9 h (banked from W12–W14)

Notes

- Review the 1-page cheat sheet (modes, coefficients, process model table). Sleep 9 h.
- COCOMO numericals are free marks if the coefficients were memorized; DFDs are free marks if levels were balanced; theory rewards structure.
- **Ledger frozen during Wk15** — exam window 30 Nov – 18 Dec is a separate phase (per `weeks/SE-W14.md` L58–60).